

fuClaw AIoT 代理人技術教學手冊

適用硬體：Realtek Ameba Pro2 系列（AMB82-mini / HUB 8735 Ultra）

作者：ChungYi Fu

GitHub：<https://github.com/fustyles/fuClaw>

摘要

fuClaw 是一套運行在 Realtek Ameba Pro2 嵌入式平台上的**多模態自主 AI Agent 框架**。它將 Google Gemini 的先進多模態能力（文字、視覺、接地搜尋）與嵌入式硬體深度融合，實現了從自然語言指令到物理硬體執行的完整智能閉環。

與傳統 IoT 裝置最大的不同在於：fuClaw 不再是被動的「執行器」，而是一個具備**感知、推理、規劃、行動、記憶與自主學習能力的混合自主代理**（Hybrid Autonomous Agent）。

核心特色：

- Prompt-Orchestrated Tool Calling 機制，結合本地 ArduinoJson 嚴格驗證
- 多通道互動（Web 配置介面 + MQTT 雙向通訊 + Telegram Bot）
- SD 卡完整持久化（對話歷史、排程任務、執行狀態）
- 硬體安全機制（GPIO 白名單、原子執行、最長有效前綴、確認機制）
- FreeRTOS 多任務架構，支援即時回應與背景自主任務
- 軟體定義行為（Configuration as Code），修改 md/json 檔案即可改變 Agent 行為

本手冊從設計哲學、系統架構、核心實現、安全機制、教學應用到技術限制，進行全面且深入的說明，適合研究人員、開發者與學生使用。

目錄

- 第 1 章 代理人核心定義與設計哲學
- 第 2 章 系統運作邏輯鏈 (Workflow Deep Dive)
- 第 3 章 軟體定義行為 (SD 卡持久化配置)
- 第 4 章 多模態感官處理與編碼技術
- 第 5 章 工具箱與硬體安全規範
- 第 6 章 自主評估與連貫工作流
- 第 7 章 MQTT 與多通道通訊架構
- 第 8 章 防盜偵測技能實作 SOP
- 第 9 章 技術侷限、效能分析與教學反思
- 第 10 章 結論與未來展望
- 附錄 A 常見錯誤排查與 FAQ
- 附錄 B 延伸實作挑戰題庫

第 1 章 代理人核心定義與設計哲學

1.1 AI Agent 的演進背景與意義

在傳統嵌入式系統中，IoT 裝置通常扮演「被動執行器」的角色。它們只能接收特定格式的指令，按照開發者預先寫好的程式邏輯執行固定動作，然後回傳結果。這類系統在本質上缺乏對自然語言的理解能力、情境推理能力、長期記憶能力，以及自主規劃多步驟任務的能力。

當面對複雜、動態或需要判斷的指令時，例如「如果外面下雨就把窗簾關上，並開啟客廳的燈，如果溫度超過 28 度就開啟風扇」，傳統 IoT 裝置幾乎無法獨立完成，需要仰賴外部 App、雲端腳本或人工介入。這導致系統延遲高、耦合度高、可擴展性差，且使用者體驗不佳。

fuClaw 的設計目標正是打破這個限制。它讓中階嵌入式裝置（Realtek Ameba Pro2 系列，包括 AMB82-mini 與 HUB 8735 Ultra）成為一個真正的 AI Agent，一個能夠：

- 理解自然語言（包含繁體中文、長句、情境描述與模糊指令）
- 分析即時視覺影像（透過相機與 Gemini Vision）
- 進行多步驟推理與規劃
- 安全地控制物理硬體（GPIO、伺服馬達、感測器等）
- 記住歷史對話與裝置狀態
- 自主執行背景任務（如定時排程與防盜偵測）

的智能實體。

這標誌著嵌入式系統從「智慧裝置」向「智能代理」的重大演進。在 2026 年，這種「邊緣端具身 AI Agent」正是嵌入式 AI 領域最受關注的發展方向之一。

1.2 現代 AI Agent 的六大核心要素

fuClaw 在資源受限的 MCU 平台上，完整實作了現代 AI Agent 的六大核心要素：

1. **對話 (Conversation)** 提供三種互動通道：
 - Web 配置與即時聊天介面 (index.html 系列)
 - MQTT 低延遲雙向通訊
 - Telegram Bot 遠端控制 使用者可根據不同場景選擇最適合的互動方式。
2. **推理 (Reasoning)** 以 Google Gemini 3 Flash Preview 作為核心推理引擎，支援文字、多模態視覺輸入與 grounded web search，能夠處理條件判斷、多步驟規劃、常識推理與即時資訊查詢。
3. **工具使用 (Tool Use)** 系統定義了 20 餘種原子工具 (atomic tools)，包括 GPIO 控制、拍照、視覺分析、網路搜尋、延遲、排程管理等。所有工具呼叫均以 JSON 格式輸出，並在本地進行嚴格的結構驗證與安全檢查。
4. **視覺感知 (Vision)** 整合板載相機，可即時擷取影像、進行快取，並透過 Gemini Vision 進行語意分析、物件偵測與情境理解。
5. **記憶 (Memory)** 使用 AmebaFatFS + SD 卡實現對話歷史、排程任務、執行狀態的完整持久化。系統在啟動時自動載入 memory.md，實現斷電後狀態自動恢復。
6. **硬體行動 (Action)** 安全控制各種物理硬體，包括 GPIO 數位/類比輸出、SG90 伺服馬達、DHT11 溫溼度感測器、警示燈等，同時具備多層安全防護機制。

這六大要素的無縫整合，讓 fuClaw 在嵌入式平台上接近實現了「具身智能 (Embodied AI)」的概念。

1.3 fuClaw 的設計哲學

1.3.1 Prompt-Orchestrated (提示詞驅動) fuClaw 不依賴 Gemini 的原生 Function Calling API，而是採用更靈活且可控的「提示詞驅動工具路由」機制。透過在系統提示詞中詳細定義工具格式、執行規則與安全限制，Gemini 會自行輸出符合規範的 JSON 工具呼叫。本地韌體再進行解析與驗證。

此設計的優點是：

- 對舊版或非原生支援 Function Calling 的模型仍有良好相容性
- 開發者對工具行為有完整控制權
- 更容易加入自訂安全規則與教學引導

1.3.2 Safety-First (安全優先) 硬體控制是嵌入式 Agent 最危險的部分。fuClaw 設計了多層安全機制：

- GPIO 白名單 (僅允許 device.md 中明確定義的腳位)
- 原子執行規則 (單次僅一個完整動作)
- 最長有效前綴規則 (多步驟請求時，遇到無效即停止)
- 使用者指令硬體動作需明確確認 (排程任務視為預授權)

1.3.3 Configuration as Code (配置即程式碼) Agent 的核心行為 (人格、裝置映射、技能) 均以 Markdown / JSON 檔案定義：

- 修改 soul.md 可改變 Agent 個性與語氣
- 修改 device.md 可新增或調整硬體映射
- 修改 skill.md 可新增複合自主技能

這大大降低了開發與教學門檻，非程式背景的使用者也能參與 Agent 的客製化。

1.3.4 Hybrid Autonomous Agent (混合自主代理) fuClaw 同時支援兩種運作模式：

- **互動模式 (Reactive)**：使用者主動下達指令，系統即時解析並回應。
- **自主模式 (Proactive)**：透過背景任務 (如 task_time_scheduling 與防盜偵測) 主動監控環境並執行預設技能。

這種混合模式讓 fuClaw 既能滿足即時控制需求，又能實現無人值守的智慧行為。

1.4 教學重點與討論題

- 請比較傳統 IoT 與 fuClaw Agent 在面對複合指令時的處理差異。
- Prompt-Orchestrated 與原生 Function Calling 的優缺點各是什麼？
- 為什麼嵌入式 Agent 必須特別強調「Safety-First」？

第 2 章 系統運作邏輯鏈 (Workflow Deep Dive)

2.1 系統整體生命週期概覽

fuClaw 的運作可分為三個主要階段，形成一個完整的閉環：

階段①：初始化階段 (setup()) 系統啟動時執行以下重要動作：

- 讀取 SD 卡中的 env.json 並套用 WiFi、MQTT、Gemini、Telegram 等設定
- 初始化相機、DHT11、SG90 伺服馬達、RTC 等硬體
- 讀取 soul.md、device.md、skill.md 建立系統提示詞
- 載入 memory.md 恢復對話歷史
- 建立 FreeRTOS 多任務
- 初始化 MQTT 連線與 Web Server

階段②：持續互動階段 系統透過以下任務持續接收外部輸入：

- task_getRequest：處理 Web 請求
- task_getMqttMessage：處理 MQTT 訊息
- task_getTelegramMessage (另一版本)：處理 Telegram Bot 訊息

階段③：自主運作階段

- task_time_scheduling：每 60 秒檢查排程任務
- (可選) task_theft_detection：背景防盜偵測技能

2.2 FreeRTOS 多任務並行架構

fuClaw 充分利用 Ameba Pro2 的雙核優勢，使用 FreeRTOS 建立多個獨立任務，確保系統同時具備即時回應能力與背景自主能力：

任務名稱	堆疊大小	主要職責
task_getRequest	16KB	Web 伺服器請求處理
task_getRequestStream	16KB	MJPEG 即時影像串流
task_getMqttMessage	32KB	MQTT 連線維護與訊息回調
task_time_scheduling	6KB	排程任務檢查與執行
task_getTelegramMessage	16KB	Telegram Bot 長輪詢（另一版本）

這種多任務設計讓系統在處理使用者即時指令的同時，仍能穩定執行背景排程與防盜偵測。

2.3 訊息完整處理流程 (Workflow Deep Dive)

以使用者透過 MQTT 發送「請開啟綠色 LED」為例，系統完整處理流程如下：

1. **訊息接收** callback() 函式接收 MQTT 訊息，產生 workId (如 <MQTT> 2026-06-08 15:30:22)。
2. **路由與提示詞組建** 系統將使用者訊息與 soul.md + device.md + devicesRule + skillsDefinition + toolsDefinition 組合成完整的系統提示詞，送至 Gemini。
3. **Gemini 推理** Gemini 根據提示詞理解意圖，輸出結構化 JSON Tool Call。
4. **回應處理 (handleAgentResponse)**
 - 清理 Gemini 可能輸出的 Markdown 標記、多餘換行、跳脫字元
 - 判斷輸出類型：
 - 單一 JSON 物件 → 直接執行
 - JSON 陣列 → 循序執行（最長有效前綴規則）
 - 純文字 → 自然語言回覆
 - "NONE" → 結束工作流程
5. **工具執行 (executeTool)** 呼叫對應函式，例如 /digitalwrite → toolPinOutput() → digitalWrite(24, 1)
6. **結果回注與狀態更新** 執行結果轉為文字，加入 historicalMessages，儲存至 SD 卡。
7. **工作流程評估** 若 reCheck = true，呼叫 evaluateWorkflowContinuation() 詢問 Gemini：「任務是否完成？是否需要繼續？」
8. **回覆使用者** 透過 MQTT / Web / Telegram 將最終結果傳送給使用者。

2.4 關鍵函式深度解析

`handleAgentResponse()` 這是整個系統的「大腦分流器」。它負責：

- 字串清理（移除 ‘‘ json
- 格式判斷（單 JSON / JSON 陣列 / 純文字 / NONE）
- 呼叫對應處理分支

`evaluateWorkflowContinuation()` 實現 OODA 循環中的 Re-Observe 階段。每次工具執行完畢後，系統會自動詢問 Gemini 是否需要繼續下一步動作。這是 fuClaw 能夠執行多步驟複雜任務的核心機制。

`executeTool()` 工具分派中心，包含嚴格的安全驗證：

- GPIO 白名單檢查
- 參數範圍驗證（digitalwrite 只能是 0 或 1）
- 原子執行保證

2.5 教學重點與討論題

1. 為什麼 `handleAgentResponse()` 需要進行字串清理？Gemini 的輸出常見哪些干擾格式？
2. 「最長有效前綴規則」在多步驟任務中的重要性是什麼？如果省略這個規則可能發生什麼問題？
3. FreeRTOS 多任務設計如何平衡即時回應與背景自主任務？如果只使用單一 `loop()` 會有什麼缺點？

第 3 章 軟體定義行為（SD 卡持久化配置）

3.1 軟體定義行為（Configuration as Code）的設計理念

fuClaw 最具教學價值與實用性的設計之一，就是將 Agent 的核心行為「軟體定義化」。傳統嵌入式專案中，改變 AI 個性、硬體映射或新增技能通常需要修改 C++ 程式碼並重新燒錄韌體，這對教學與快速迭代非常不友善。

fuClaw 則採用 **Configuration as Code** 的現代理念：將 Agent 的人格、硬體定義、技能流程、對話記憶 等核心行為抽離成 SD 卡上的文字檔案。使用者（或學生）只需修改 Markdown 或 JSON 檔案，重新啟動裝置即可生效，無需重新燒錄韌體。

這種設計大幅降低了門檻，讓非程式背景的使用者也能參與 Agent 的客製化，同時提升了系統的可維護性與可擴展性。

3.2 五大核心設定檔詳細說明

fuClaw 在 SD 卡中維護以下主要設定檔案，每個檔案都有明確的職責：

檔案名稱	格式	主要用途	重要性
env.json	JSON	系統環境變數（裝置名稱、WiFi、MQTT、Gemini 金鑰等）	★★★★★
soul.md	Markdown	Agent 人格與行為規範	★★★★★
device.md	Markdown	硬體映射白名單與安全規則	★★★★★
skill.md	Markdown	高階複合技能定義	★★★★☆
memory.md	JSON 片段	對話歷史持久化	★★★★☆
schedule.json	JSON	排程任務清單	★★★★☆
scheduleTodayExecuted.md	文字	今日已執行循環任務記錄	★★★★☆☆

3.2.1 env.json —— 系統環境設定

這是系統啟動時第一個被讀取的檔案，內容範例：

JSON

```
{  
  "device_name": "你的裝置名稱",  
  "wifi_ssid": "你的WiFi 名稱",  
  "wifi_pass": "你的WiFi 密碼",  
  "mqtt_server": "mqttgo.io",  
  "mqtt_port": 1883,  
  "mqtt_user": "",  
  "mqtt_password": "",  
  "mqtt_subscribeTextTopic": "fuclaw/subscribe",  
  "gemini_apikey": "AIzaSyxxxxxxxxxxxxxxxx",  
  "gemini_model": "gemini-3-flash-preview",  
  "schedule_timeout": 10,    //分鐘  
  "timezone": "Asia/Taipei"  
}
```

系統在 `setup()` 中透過 `setEnvironmentSettings()` 解析並套用這些設定。

3.2.2 soul.md —— Agent 人格定義

這是影響 Agent 語氣、決策風格與角色設定的核心檔案。內容會直接覆蓋程式碼中的 `geminiRole` 變數。

範例 1：親切家庭助理

Markdown

你是一個溫暖、細心且活潑的家庭助理。使用繁體中文回答，語氣輕鬆親切，適時加入鼓勵的話語。遇到硬體控制請求時，務必先確認使用者意願。

範例 2：嚴謹科學實驗助理

Markdown

你是一位嚴謹的科學實驗助理。回答時必須使用客觀語言、引用數據，並在任何硬體操作前提醒安全注意事項。

3.2.3 device.md —— 硬體映射與安全白名單

這是 `fuClaw` 安全機制的核心檔案。只有在此檔案中明確定義的裝置才能被控制。

範例內容：

Markdown

```
=====
已確認硬體裝置
=====
```

HUB 8735 Ultra

- 綠色 LED：腳位 25
- 藍色 LED：腳位 26
- 補光 LED：腳位 13（類比輸出 0-255）
- 功能按鈕：腳位 12（僅輸入，主動低電位）

外部模組

- SG90 伺服馬達（窗戶）：腳位 12（角度 0-180）
- DHT11 溫溼度感測器：腳位 20
- 緊急按鈕：腳位 1（主動高電位）

3.2.4 skill.md —— 高階技能定義

用來定義複合型自主技能，例如內建的防盜偵測。

範例（防盜偵測）：

Markdown

```
=====
SKILL: anti_theft_detection
=====
```

Goal: 偵測人類存在並觸發警示流程

MUST OUTPUT EXACT JSON ARRAY ONLY

Step 1: Vision 分析

```
{
  "type": "tool_call",
  "method": "/vision",
  "params": {
    "query": "Determine whether a person is visible in the image.",
    "frames": true,
    "task": "If person detected, continue. Else return NONE."
  }
}
```

3.2.5 memory.md —— 對話歷史持久化

儲存完整的 Gemini 對話歷史（JSON messages 陣列格式）。系統重啟後自動載入，實現「記憶不遺失」。

3.3 熱更新機制

fuClaw 在 setup() 中會自動讀取這些檔案：

- 修改後只需重新啟動裝置（/reboot 或斷電重開）
- 無需修改 C++ 程式碼即可改變 Agent 行為

這是 fuClaw 在教學上極具優勢的設計。

3.4 教學重點與討論題

1. 為什麼要把 Agent 行為抽離成獨立檔案，而不是寫死在程式碼中？
2. 如果 device.md 中沒有定義某個 GPIO，AI 要求控制它時會發生什麼事？這對安全有什麼意義？
3. 如何設計一個新的技能，讓 Agent 每小時自動回報室內溫溼度？

第 4 章 多模態感官處理與編碼技術

4.1 多模態感知的重要性

傳統嵌入式 IoT 裝置通常只具備單一感知管道（例如僅支援文字指令或單一感測器）。fuClaw 則大幅提升感知能力，整合視覺（Vision）與語音（Speech）兩種主要感官管道，讓 Agent 能夠像人類一樣「看」與「聽」，大幅提升情境理解能力。

本章將詳細說明 fuClaw 如何在資源受限的 MCU 上實現多模態感知、影像與音訊的編碼技術、快取機制，以及相關教學重點。

4.2 視覺處理流程（Vision Pipeline）

fuClaw 的視覺能力主要透過兩個工具實現：

- `/still`：單純拍照並傳送影像
- `/vision`：拍照 + Gemini 多模態分析

完整處理流程：

1. **影像擷取** 使用 `Camera.getImage()` 從板載相機取得 320×240 JPEG 影像。
2. **影像快取機制** 系統使用全域變數 `imageAddress` 與 `imageLength` 儲存最新影像資料。
 - `frames: true` → 重新擷取新畫格
 - `frames: false` → 使用上次快取的影像（避免重複擷取，提高效率）
3. **Base64 編碼** 因為 Gemini API 接受 JSON 格式，必須將二進位 JPEG 轉換為 Base64 字串才能嵌入請求中。

核心程式碼片段：

C++

```
String imageFile = "";
uint8_t *fbBuf = (uint8_t*)img_addr;
size_t fbLen = img_len;

for (size_t i = 0; i < fbLen; i += 3) {
    char output[4];
    base64_encode(output, (char*)(fbBuf + i), 3);
    imageFile += String(output);
}
```

4. 送至 Gemini Vision 將 Base64 字串嵌入 JSON 的 inline_data 欄位，連同使用者查詢一起送出：

JSON

```
{
  "contents": [{
    "parts": [
      { "text": "Determine whether a person is visible in the image." },
      {
        "inline_data": {
          "mime_type": "image/jpeg",
          "data": "BASE64_STRING_HERE"
        }
      }
    ]
  }
]
```

}

4.3 /still 與 /vision 的使用策略

工具	適用情境	frames 參數建議	特點
/still	使用者單純想要看目前畫面	true	直接傳送影像，不做 AI 分析
/vision	需要 AI 理解影像內容	true	進行語意分析，可觸發後續動作
/still (快取)	搭配 /vision 後傳送「當時分析的影像」	false	確保警示圖片與偵測時間點一致

設計考量：在防盜偵測技能中，先用 /vision 分析是否有人的人同時快取影像，若偵測到人，再用 /still (frames:false) 傳送「當時那張有人」的影像，避免後續新畫格中人已離開導致警示失效。

4.4 語音處理流程 (Speech-to-Text)

當使用者透過 Telegram 傳送語音訊息時，fuClaw 執行以下完整流程：

1. **取得檔案路徑** 呼叫 Telegram getFile API 取得語音檔案 CDN 路徑 (OGG/Opus 格式)。
2. **下載音訊** 使用 HTTP/1.0 下載原始二進位資料至記憶體緩衝區 (最大 256 KB)。
3. **Base64 編碼** 將音訊轉為 Base64，嵌入 Gemini 的 inline_data 欄位。
4. **語音轉文字** 送至 Gemini 進行 Speech-to-Text，取得文字後視為一般文字訊息，進入標準 Gemini 推理流程。

為什麼使用 HTTP/1.0？ HTTP/1.1 的 Chunked Transfer Encoding 會在資料前加入長度標頭，嵌入式系統處理較複雜。HTTP/1.0 直接傳送純二進位本體，大幅簡化接收邏輯。

4.5 效能與資源考量

- **影像處理成本**：單張 320×240 JPEG Base64 後約 20 - 55 KB，傳輸與 API 處理時間約 5 - 12 秒。
- **記憶體使用**：Base64 編碼過程需要較大暫存空間，系統會在處理前檢查可用 heap。
- **優化建議**：定期執行 /reset 清除過長對話歷史；背景任務中避免同時進行視覺分析。

4.6 教學重點與討論題

1. 為什麼 fuClaw 在 /vision 後使用 frames: false 傳送影像？這對任務一致性有什麼幫助？
2. Base64 編碼雖然方便，但會增加 33% 的資料量。在嵌入式環境中，這帶來什麼 trade-off？
3. 如果要讓 Agent 「聽懂」語音指令並控制硬體，需要經過哪些處理步驟？哪一步最容易出錯？
4. 如何設計一個技能，讓 Agent 先用語音確認使用者意願，再執行硬體動作？

第 5 章 工具箱與硬體安全規範

5.1 工具系統設計理念

fuClaw 的工具系統是整個 Agent 能夠安全且可靠地與物理世界互動的核心。與一般 LLM 直接輸出文字描述硬體動作不同，fuClaw 採用**原子工具呼叫 (Atomic Tool Calling)** 機制，強制 Gemini 只能輸出結構化 JSON，並在本地進行嚴格驗證後才執行硬體操作。

這種設計的核心目標是 **Safety-First**：防止模型幻覺導致危險的硬體誤操作，同時保持高度的可擴展性。

5.2 完整工具清單 (2026-06-08 版)

工具名稱	類型	主要功能	安全等級
/digitalwrite	硬體輸出	設定 GPIO 為 0 或 1	高
/analogwrite	硬體輸出	PWM 輸出 0 - 255	高
/digitalread	硬體輸入	讀取數位腳位狀態	中
/analogread	硬體輸入	讀取類比數值	中
/still	視覺	拍照並傳送影像	低
/vision	視覺 + 推理	拍照 + Gemini 多模態分析	低
/search	外部知識	Google 接地搜尋	低
/delay	系統控制	暫停指定毫秒	中
/getMemory	系統診斷	顯示記憶體使用狀況	低
/getLog	系統診斷	顯示工具執行歷史	低
/reset	系統控制	清空對話歷史	高
/reboot	系統控制	重新啟動裝置	極高

/schedule	排程	新增排程任務	高
/getSchedule	排程	取得所有排程	低
/updateScheduleStatus	排程	更新執行狀態	中
/tcpSendMessage	通訊	透過 TCP 傳送訊息給另一個裝置或代理 (Agent)	中
/mqttSendMessage	通訊	透過 TCP 或 MQTT 傳送訊息給另一個裝置、代理 (Agent) 或所有 MQTT 訂閱者	中
/mqttSendImage	通訊	透過 MQTT 傳送影像快照 (Video Snapshot) 給另一台 fuClaw 裝置或所有 MQTT 訂閱者	中
/telegramSendMessage	通訊	傳送訊息到 Telegram Bot	中
/telegramSendImage	通訊	傳送影像到 Telegram Bot	中
/lineSendMessage	通訊	傳送訊息到 LINE Bot	中

5.3 原子執行規則 (Atomic Execution Rule)

這是 fuClaw 安全設計的最核心原則：

- **單一腳位**：每次 tool_call 只能操作一個 GPIO
- **單一操作**：不能在同一個 tool_call 中同時讀取與寫入
- **單一數值**：digitalwrite 只能是 0 或 1，不能是 "blink" 或複合指令
- **單一步驟**：一個回應原則上只執行一個完整工具

正確的多步驟範例 (LED 閃爍一次)：

JSON

```
[
  {
```

```
[
  {
    "type": "tool_call",
    "method": "/digitalwrite",
    "params": { "pin": 26, "pinmode": "digitalwrite", "value": 1 }
  },
  {
    "type": "tool_call",
    "method": "/delay",
    "params": { "milliseconds": 500 }
  },
  {
    "type": "tool_call",
    "method": "/digitalwrite",
    "params": { "pin": 26, "pinmode": "digitalwrite", "value": 0 }
  }
]
```

5.4 最長有效前綴規則 (Longest Valid Prefix)

當 Gemini 輸出多個 tool_call 組成的 JSON 陣列時，系統不會「跳過錯誤項目」，而是採用**最長有效前綴**策略：

- 從頭開始逐一檢查每個 tool_call
- 一旦遇到無效或不完整的 tool_call，立即停止執行，後續所有項目全部捨棄
- 這有效防止了在不完整計畫下執行危險動作

此規則大大提升了系統安全性。

5.5 確認機制 (Global Device Control Policy)

為了防止 AI 擅自修改硬體狀態，fuClaw 設有以下確認規則：

- **一般使用者指令**：所有會改變硬體狀態的動作（digitalwrite、analogwrite、reboot 等）必須先取得使用者明確確認
- **排程任務與技能**：視為預先授權，可直接執行（例如防盜偵測中的 LED 警示）
- **政策變更**：若使用者要求「以後不用確認」，系統必須再次確認此變更

5.6 數值驗證與邊界處理

- digitalwrite：只能接受 0 或 1，否則拒絕執行
- analogwrite：使用 constrain() 強制限制在 0 - 255
- delay：限制在 0 - 10000 毫秒，防止無限等待
- 所有 GPIO 操作前必須通過 device.md 白名單檢查

5.7 教學重點與討論題

1. 為什麼要強制「原子執行」？如果允許 Gemini 一次輸出多個動作，可能發生什麼風險？
2. 「最長有效前綴規則」與「跳過錯誤項目」哪一種更安全？為什麼？
3. 在實際教學中，如何引導學生設計一個安全的複合技能（例如「溫度過高時自動開風扇並通知使用者」）？
4. 確認機制的設計對工業應用有什麼重要意義？

第 6 章 自主評估與連貫工作流

6.1 為什麼需要自主評估機制？

傳統聊天機器人通常採用「一問一答」模式，使用者每發送一條訊息，系統就回應一次，然後等待下一條指令。這種模式在處理簡單任務時足夠，但在執行多步驟複雜任務（如「偵測是否有人 → 如果有人就拍照並閃爍警示燈三次 → 完成後回報」）時就會顯得斷斷續續，需要使用者不斷介入。

fuClaw 設計了**自主評估與連貫工作流**機制，讓 Agent 能夠在執行工具後，主動判斷「任務是否完成？是否需要繼續下一步？」這是 fuClaw 從「被動聊天機器人」進化為「自主代理」的關鍵能力之一。

6.2 OODA 循環在 fuClaw 中的實現

fuClaw 的自主工作流借鑑了軍事決策理論中的 OODA 循環（Observe - Orient - Decide - Act）：

OODA 階段	fuClaw 對應元件	具體實現
Observe	/vision、/digitalread、/analogread	感知環境狀態
Orient	Gemini 推理引擎	結合歷史記憶與目前感知進行分析
Decide	JSON Tool Call 輸出	決定下一步動作
Act	executeTool()	執行硬體或系統動作
Re-Observe	evaluateWorkflowContinuation()	評估結果並決定是否繼續

這個循環讓 Agent 能夠在多步驟任務中自主推進，而不需要使用者每一步都介入。

6.3 evaluateWorkflowContinuation() 核心機制

這是 fuClaw 實現連貫工作流的關鍵函式。在每個工具執行完畢後（若 reCheck = true），系統會自動呼叫此函式：

```
C++
void evaluateWorkflowContinuation(String workId, bool reCheck, String task = "") {
    if (!reCheck) return;

    String prompt =
        "This is a deterministic workflow evaluation step.\n"
        "Analyze the execution result and current state.\n"
        "If the workflow is complete, return EXACTLY: NONE\n"
        "If additional actions are needed, return ONLY tool_call JSON.\n"
        "Do not explain. Do not add natural language.";
```

```
String response = geminiChatRequest(workId, prompt);
handleAgentResponse(workId, response);
}
```

設計重點：

- 只在最後一個工具執行後才進行評估（避免中間步驟頻繁呼叫 Gemini，節省 Token 與時間）
- 強制 Gemini 輸出 "NONE" 作為完成哨兵
- 確保輸出格式乾淨（僅 JSON 或 NONE）

6.4 reCheck 旗標設計邏輯

在 JSON 陣列執行過程中，每個 tool_call 都帶有 reCheck 參數：

位置	reCheck 值	行為說明
中間的 tool_call	false	直接執行下一個工具，不進行評估
最後一個 tool_call	true	執行完畢後呼叫 evaluateWorkflowContinuation()

這種設計有效平衡了執行效率與自主判斷能力。

6.5 "NONE" 哨兵機制

當 Gemini 判斷整個工作流程已完成時，必須**精確回傳**字串 "NONE"（全大寫，無多餘文字）。系統在 handleAgentResponse() 中會明確檢查此值並終止流程。

常見錯誤示範：

- Gemini 回傳 "None" 或 "工作已完成" → 系統無法識別，會當作普通文字回覆
- 解決方式：在 skill.md 和系統提示詞中反覆強調「必須回傳精確的 NONE」

6.6 任務鏈 (Task Chaining) 完整範例

範例任務：「搜尋今日高雄天氣，若下雨則關閉補光燈並通知使用者」

1. /search "高雄今日天氣"
2. evaluateWorkflowContinuation() → Gemini 判斷有雨，繼續
3. 使用者確認後 → /analogwrite pin=13 value=0 (關燈)
4. evaluateWorkflowContinuation() → Gemini 判斷完成，回傳 "NONE"

6.7 教學重點與討論題

1. OODA 循環在嵌入式 Agent 中的意義是什麼？與傳統程式流程有何不同？
2. 為什麼要設計 reCheck 旗標？如果每個工具都呼叫 evaluateWorkflowContinuation() 會有什麼問題？
3. 「NONE」哨兵機制為什麼要強制全大寫且不允許額外文字？這對系統穩定性有何幫助？
4. 如何設計一個更複雜的自主技能，例如「溫度超過 30 度時開風扇，並每 5 分鐘回報一次狀態，直到溫度低於 28 度」？

第 7 章 MQTT 與多通道通訊架構 (2026-06-08 重要更新)

7.1 多通道並存設計理念

2026-06-08 版本的最大升級之一，是 fuClaw 完整支援**多通道並存架構**。系統不再依賴單一通訊方式，而是同時提供 Web、MQTT 與 Telegram 三種通道，讓 Agent 能夠適應不同使用場景：

- **Web 通道：**適合本地配置、管理與即時聊天（適合教學展示與開發除錯）
- **MQTT 通道：**適合低延遲、事件驅動、裝置間通訊（適合智慧家居、工業監控）
- **Telegram 通道：**適合遠端行動控制與通知（適合個人使用者）

三種通道可同時運行，互不干擾，這大大提升了 fuClaw 的實用性與可擴展性。

7.2 MQTT 通訊架構詳細說明

MQTT 是 fuClaw 實現低延遲與裝置間協作的核心通道。

7.2.1 主題設計

- 訂閱主題 (Subscribe): xxxxxxxxxx/subscribe —— 接收來自外部系統或使用者的指令
- 發佈文字主題 (Publish): xxxxxxxxxx/publish —— 傳送文字回應
- 發佈影像主題 (Publish): xxxxxxxxxx/publishimage —— 傳送 JPEG 影像 (Base64 或二進位)

7.2.2 核心程式碼解析

C++

// MQTT 回調函式

```
void callback(char* topic, byte* payload, unsigned int length) {  
    String workId = "<MQTT> " + getRtcTimeString();
```

// 將 payload 轉為 String

```
char* message = (char*)malloc(length + 1);  
memcpy(message, payload, length);  
message[length] = '\0';
```

```
String text = String(message);

if (text.startsWith("/")) {
    executeTool(workId, text, JsonObject());
} else {
    text = geminiChatRequest(workId, text);
    handleAgentResponse(workId, text);
}

free(message);
storeDataToFile(memoryFilename, historicalMessages);
}
```

重要特性：

- 使用非阻塞模式 (wifiClient.setNonBlockingMode())
- 自動重連機制 (reconnect() 函式)
- 支援 /help、slash command 與自然語言混合輸入

7.3 Web + MQTT + Telegram 三通道協同工作

- Web 介面 (index_mqtt_chat.html) 提供美觀的 MQTT 聊天面板
- MQTT 可與外部系統 (如 Home Assistant、Node-RED) 無縫整合
- Telegram 適合推送通知與遠端控制
- 三者共用同一套 Gemini 推理引擎與工具系統

7.4 實際應用場景

1. **智慧家居集群**：多台 fuClaw 裝置透過 MQTT 互相通訊，組成分散式 Agent 網路
2. **工業監控**：感測器數據透過 MQTT 上傳中央系統
3. **教育示範**：學生可同時使用 Web 配置、MQTT 測試與 Telegram 遠端控制
4. **遠端照護**：家長可透過 Telegram 接收兒童房間影像與環境狀態

7.5 安全性與效能考量

- MQTT 可設定使用者名稱與密碼
- 所有硬體控制指令仍受本地 device.md 白名單與確認機制保護
- 使用 QoS 0 降低延遲，適合即時控制
- 背景任務中會暫停 Telegram 輪詢，避免網路資源衝突

7.6 教學重點與討論題

1. MQTT 與 HTTP Web 相比，有什麼優缺點？在什麼場景適合使用 MQTT？
2. 如何讓兩台 fuClaw 裝置透過 /agentSendMessage 互相協作（例如一台偵測到人，另一台開燈）？
3. 如果 MQTT Broker 斷線，系統如何恢復？這對可靠性的影響是什麼？
4. 設計一個跨裝置技能：客廳裝置偵測到有人，就通知臥室裝置開燈。

第 8 章 防盜偵測技能實作 SOP

8.1 技能概述

技能名稱：anti_theft_detection（防盜偵測） **觸發方式：**由背景任務 task_theft_detection 每 5 分鐘自動執行（自主模式） **目標：**偵測相機畫面中是否有人類，若偵測到則傳送警示照片並閃爍藍色 LED 三次 **適用裝置：**HUB 8735 Ultra（藍色 LED 腳位 26） **豁免確認：**是（系統自主執行技能，不需使用者每次確認）

這是 fuClaw 內建的經典自主技能範例，充分展示了**視覺感知** + **推理** + **硬體行動** + **工作流連貫**的完整能力。

8.2 技能執行流程逐步解析

步驟	工具呼叫	詳細說明與決策邏輯
0	evaluateWorkflowContinuation	背景任務注入提示詞，要求執行 anti_theft_detection 技能
1	/vision (frames: true)	擷取最新影像，詢問 Gemini：「圖中是否有人類？」
判斷 A	返回 "NONE"	Gemini 判斷無人，任務結束
判斷 B	繼續執行警示序列	Gemini 確認有人，進入警示工作流
2a	/still (frames: false)	傳送 Step 1 當時快取的「有人」影像至 Telegram
2b	/digitalwrite pin=26 value=1	開啟藍色 LED
2c	/delay 500ms	等待 0.5 秒
2d	/digitalwrite pin=26 value=0	關閉藍色 LED
2e	/delay 500ms	等待 0.5 秒（重複 2b-2e 共三次，形成閃爍效果）
Final	evaluateWorkflowContinuation	Gemini 確認三次閃爍完成，回傳 "NONE" 結束技能

8.3 為什麼 /still 使用 frames: false？

這是技能設計的重要細節：

- Step 1 的 /vision 已擷取並快取了「當時有人」的影像
- 若 Step 2a 改用 frames: true 重新拍照，可能此時人已離開畫面，導致警示照片與偵測結果不一致
- 使用 frames: false 可確保警示照片與 Gemini 判斷的時間點一致，提升可靠性

8.4 完整的 skill.md 定義範例

Markdown

```
=====
SKILL: anti_theft_detection
=====
```

Goal：偵測人類存在並觸發警示工作流程

MUST OUTPUT EXACT JSON ARRAY ONLY

Step 1: Vision 分析

```
{
  "type": "tool_call",
  "method": "/vision",
  "params": {
    "query": "Determine whether a person is visible in the image.",
    "frames": true,
    "task": "If person detected, continue workflow. Else return NONE."
  }
}
```

Step 2: 警示序列（有人時輸出此陣列）

```
[
  { "type":"tool_call", "method":"/still", "params":{"frames":false, "task":"NONE"} },
  { "type":"tool_call", "method":"/digitalwrite", "params":{"pin":26, "pinmode":"digitalwrite", "value":1} },
  { "type":"tool_call", "method":"/delay", "params":{"milliseconds":500} },
  { "type":"tool_call", "method":"/digitalwrite", "params":{"pin":26, "pinmode":"digitalwrite", "value":0} },
  { "type":"tool_call", "method":"/delay", "params":{"milliseconds":500} }
  // 重複三次閃爍...
]
```

8.5 技能擴充挑戰（教學實作建議）

初級：修改閃爍次數為 5 次，並在結束後傳送文字通知「警告：偵測到可疑人員」。

中級：新增光線判斷。先用 /analogread 讀取光感測器，若環境明亮（白天）才進行視覺偵測，減少夜間誤報。

高級：結合溫度感測器，設計「異常高溫 + 有人」雙條件警示技能。

創意：設計「來客問候」技能，偵測到人臉後使用 `/chat` 傳送個性化問候語，並拍照存檔。

8.6 教學重點與討論題

1. 為什麼防盜技能要使用背景任務自動觸發，而不是等待使用者指令？
2. 在 Step 2a 使用 `frames: false` 的設計優點是什麼？如果改成 `true` 會有什麼風險？
3. 如何在 `skill.md` 中加入「如果連續三次偵測到人，就觸發更強烈的警示（例如蜂鳴器）」的邏輯？
4. 自主技能的設計如何平衡「主動性」與「誤報率」？

第 9 章 技術局限、效能分析與教學反思

9.1 認識邊界：優秀工程師的必備素養

fuClaw 雖然在嵌入式平台上實現了相當完整的 AI Agent 功能，但作為一個運行在 MCU 上的系統，必然存在多方面的技術限制。真正的工程師不僅要了解系統「能做什么」，更要深刻理解「不能做什么」以及「為什麼」。本章將系統性地分析 fuClaw 的主要限制，並提供教學反思與改進建議。

9.2 主要技術局限

9.2.1 記憶體管理與堆積碎片化

問題描述：

- fuClaw 大量使用 Arduino String 類型處理對話歷史、JSON 資料與 Base64 編碼。
- 長時間運行後，頻繁的 String 拼接與 malloc/free 操作容易造成堆積碎片化（Heap Fragmentation）。
- 當可用連續記憶體不足時，即使總空餘空間足夠，也可能出現 malloc() 失敗（例如 Base64 編碼或影像緩衝區無法分配）。

監控方法：

- 使用 /getMemory 工具查看目前剩餘 heap 與歷史最低值。
- 當 xPortGetMinimumEverFreeHeapSize() 趨近 0 時，需立即處理。

緩解策略：

- 定期執行 /reset 清空對話歷史
- 減少 geminiMaxOutputTokens 設定
- 在關鍵操作前手動釋放暫存變數
- 未來可考慮實作記憶體壓縮或固定大小緩衝區

9.2.2 單一檔案過大與可維護性

目前狀況：

- 主程式 AmebaPro2_fuClaw_MQTT_SD_SG90_DHT11.ino 單檔超過 12 萬字元。
- 全域變數偏多，重複程式碼存在，長期維護困難。

建議：

- 將系統拆分成多個 .h / .cpp 檔案（例如 Tool 模組、Prompt 模組、Persistence 模組）
- 引入更完整的模組化設計

9.2.3 完全依賴雲端 Gemini

限制：

- 必須持續連網才能運作
- Gemini API 有速率限制與費用
- 網路不穩定時，系統回應會明顯延遲或失敗

未來方向：

- 加入本地小型語言模型作為離線備援
- 實作混合推理機制（雲端優先，本地後備）

9.2.4 效能瓶頸

操作	典型耗時	主要原因
/vision 分析	6 - 15 秒	影像擷取 + Base64 + HTTPS + API 處理
MQTT + Gemini 推理	2 - 6 秒	網路延遲 + Token 處理
長對話歷史	記憶體壓力增加	歷史累積導致 heap 使用上升

9.3 教學反思與討論題

1. **記憶體碎片化**：請用白紙與便利貼模擬 heap 分配過程，說明為什麼「總空餘空間足夠」卻仍可能分配失敗？
2. **安全 vs 便利**：確認機制雖然安全，但會降低使用者體驗。在什麼場景下應該豁免確認？如何設計更智慧の確認策略？
3. **單檔 vs 多檔**：為什麼目前採用單一巨大 .ino 檔案？如果要重構，應該優先拆分哪些模組？
4. **雲端 vs 本地**：完全依賴雲端 LLM 的優缺點是什麼？在工業應用中，這會帶來什麼風險？
5. **自主性邊界**：Agent 自主執行技能時，應該設定什麼樣的安全上限？例如防盜偵測連續誤報 5 次後是否應該暫停？

9.4 未來改進建議（給開發者與研究者）

- 實作記憶體壓縮機制（定期總結對話歷史）
- 支援本地小型模型（如 quantized TinyLlama 或 Gemma）
- 增加單元測試與錯誤注入測試
- 開發視覺化除錯工具（Web 即時監控 heap 使用量）
- 建立 Agent 集群協作框架（多台 fuClaw 透過 MQTT 合作）

結語：fuClaw 雖然仍有許多技術限制，但它在嵌入式平台上實現了相當完整的 Agent 能力。這些限制正是未來研究與改進的最佳切入點。希望透過本章，學生與研究者能養成「既看到亮點，也正視邊界」的工程思維。

第 10 章 結論與未來展望

10.1 全書總結

fuClaw 專案成功在資源受限的 Realtek Ameba Pro2 嵌入式平台上，建構了一個功能完整、架構清晰、安全可靠的多模態自主 AI Agent 框架。

透過 Prompt-Orchestrated Tool Calling 機制、FreeRTOS 多任務架構、SD 卡持久化設計 與 嚴格的安全規則，fuClaw 將 Google Gemini 的強大推理能力與物理硬體緊密結合，實現了從自然語言理解到實際硬體執行的完整閉環。

本手冊從設計哲學、系統架構、工具系統、多模態處理、自主工作流、MQTT 通訊，一直到教學應用與技術限制，進行了系統性且深入的說明。希望透過這份文件，能夠幫助更多研究者、開發者與學生理解如何在嵌入式平台上實作真正的 AI Agent。

fuClaw 的核心價值在於：

- 它證明了 Prompt Engineering + 本地安全驗證可以在 MCU 上實現可靠的 Agent 行為。
- 它提供了 Configuration as Code 的實踐範例，大幅降低開發與教學門檻。
- 它在安全、持久化、多通道、自主性等多個面向都達到了實用級別。

10.2 研究與教育意義

fuClaw 不僅是一個功能完整的專案，更是一個優秀的**教學與研究平台**。它涵蓋了嵌入式系統、Prompt Engineering、多模態處理、即時作業系統、安全機制、軟體定義行為等多個重要領域，非常適合：

- 大學嵌入式系統、人工智慧、IoT 相關課程
- 學生專題與畢業設計
- 智慧家居、工業監控、教育機器人等應用研究
- 開源社群共同開發與擴展

10.3 未來展望

雖然 fuClaw 已達到相當成熟的程度，但仍有許多值得繼續深耕的方向：

1. **本地化推理**：整合小型本地語言模型（如 quantized Gemma 或 TinyLlama），實現離線運作能力。
2. **記憶體優化**：開發對話總結與壓縮機制，解決長期運行下的 heap 碎片化問題。
3. **多 Agent 協作**：透過 MQTT 實現多台 fuClaw 裝置的群體智能。
4. **視覺強化**：加入物件追蹤、人臉辨識等更高階視覺能力。
5. **程式碼重構**：將單一巨大 .ino 檔案拆分成模組化庫，提升可維護性。
6. **開放平台**：建立完整的 SDK 與擴展規範，讓更多開發者參與貢獻。

10.4 最終寄語

fuClaw 的開發過程證明：即使在運算資源有限的嵌入式平台上，只要結合良好的系統設計、嚴謹的安全思維與創新的 Prompt Engineering，依然能夠創造出具有真正智能的 Agent。

我們期待更多研究者與學生在 fuClaw 的基礎上繼續探索嵌入式 AI 的無限可能。

未來已來，智能就在你的手中。

附錄 A 常見錯誤排查與 FAQ

本附錄整理了 fuClaw 在實際使用與教學過程中最常遇到的問題、錯誤訊息與解決方法，適合學生、開發者與教師快速參考。

A.1 常見錯誤訊息對照表

錯誤訊息 / 現象	可能原因	解決方法
JSON parse failed	Gemini 輸出格式不標準（包含 Markdown、額外文字）	輸入「Continue」讓 Gemini 重試；檢查 soul.md 是否過度強調格式；降低 geminiTemperature
Connection failed	WiFi 斷線、API Key 無效、Gemini 伺服器忙碌	檢查 WiFi 訊號；確認 API Key 有效；執行 /reboot 重啟裝置

Previous image does not exist	使用 frames: false 但尚未有快取影像	先執行一次 /still 或 /vision (frames:true) 建立快取
Malloc failed for Base64 encoding	堆積記憶體不足 (Heap Fragmentation)	立即執行 /reset 清空對話歷史；使用 /getMemory 監控
Gemini did not respond	API 請求逾時 (預設 20 秒)	改善網路環境；減少影像解析度；確認 API 配額
Invalid digital value	digitalwrite 的 value 不是 0 或 1	檢查 Gemini 輸出；加強 devicesRule 中的驗證提示
Schedule JSON parse failed	schedule.json 格式錯誤	使用 Web 介面重新編輯排程，或執行 /getSchedule 查看內容
Heap memory low	對話歷史過長或影像處理頻繁	定期 /reset；減少 geminiMaxOutputTokens
MQTT disconnected	Broker 斷線或網路不穩	檢查 MQTT 設定；系統會自動重連

A.2 常見教學問題解答 (FAQ)

Q1：修改了 soul.md 或 device.md，但 Agent 行為沒有改變？ A：修改 SD 卡檔案後必須**重新啟動裝置**（輸入 /reboot 或斷電重開）。系統只在 setup() 階段讀取這些檔案。

Q2：為什麼 /log 沒有在 Telegram 顯示詳細工具歷史？ A：工具執行歷史可能很長，超過 Telegram 單則訊息上限（4096 字元）。系統設計只在 Serial Monitor（序列埠監視器）顯示完整記錄。如需 Telegram 查看，可修改程式碼只傳送最近 N 筆。

Q3：如何在不重新燒錄韌體的情況下新增新的硬體裝置？ A：只需在 device.md 中新增裝置描述與 GPIO 腳位，然後重新啟動裝置即可。系統會自動讀取並套用新定義。

Q4：影像分析 (/vision) 經常失敗或很慢？ A：可能原因包括網路不穩、影像過大、API 配額用完。建議：

- 使用較低解析度 (320×240)
- 確保 WiFi 訊號良好
- 定期執行 /reset 釋放記憶體

Q5：排程任務沒有在指定時間執行？ A：請確認：

- RTC 時間是否正確（執行 `/syncrtc`）
- `schedule.json` 格式是否正確
- 任務是否已被標記為已執行（循環任務會檢查 `scheduleTodayExecuted.md`）

Q6：系統運行一段時間後變得非常慢或無回應？ A：最常見原因是堆積記憶體碎片化。解決方式是定期執行 `/reset` 清空對話歷史，或重新啟動裝置。

Q7：如何讓 Agent 「記住」之前的設定或偏好？ A：對話歷史已自動儲存在 `memory.md`。只要不執行 `/reset`，Agent 就會記住之前的互動內容。

Q8：MQTT 與 Telegram 可以同時使用嗎？ A：可以。三種通道（Web、MQTT、Telegram）設計為並行運作，互不衝突。

A.3 除錯技巧建議

1. **開啟 Serial Monitor**：這是最佳除錯工具，可看到詳細的工具執行過程與錯誤資訊。
2. **使用 `/getMemory`**：定期檢查堆積記憶體使用量。
3. **使用 `/getLog`**：查看最近的工具執行記錄。
4. **測試小步驟**：新增技能時，先測試單一步驟，再組合多步驟。
5. **備份 SD 卡**：在大幅修改前，先備份所有 `.md` 與 `.json` 檔案。

附錄 B 延伸實作挑戰題庫

本附錄收集了從基礎到專題等不同難度的實作挑戰，適合教師作為作業、學生作為專題練習，或研究者作為延伸實驗使用。所有挑戰均基於 `fuClaw` 現有架構，無需修改核心 C++ 程式碼，主要透過修改 `soul.md`、`device.md`、`skill.md` 與 `schedule.json` 來完成。

B.1 基礎挑戰（適合第 1-2 週）

1. **LED 控制進階** 修改 `skill.md`，設計一個「彩色燈光秀」技能：讓藍色與綠色 LED 交替閃爍 5 次，每次間隔 300 毫秒。
2. **人格切換** 建立兩個不同的 `soul.md` 檔案（例如「活潑版」與「嚴謹版」），透過 Web 介面快速切換 Agent 個性，並測試回答同一問題時的差異。

3. **安全機制測試** 在 device.md 中故意新增一個不存在的腳位（例如 pin 99），然後要求 Agent 控制它，觀察系統的安全阻擋機制。
4. **簡單排程** 使用 Web 排程介面新增一個每天早上 8:00 自動開啟綠色 LED 的任務，並驗證 schedule.json 的變化。
5. **影像快取測試** 連續呼叫 /vision 與 /still (frames:false)，驗證是否能正確傳送「當時分析到有人」的影像。

B.2 進階挑戰（適合第 3-5 週）

6. **環境感知技能** 設計一個新技能：每 10 分鐘讀取 DHT11 溫溼度，若溫度超過 28°C 則開啟警示燈（analogwrite pin=13, value=255），並傳送通知。
7. **條件式防盜** 改進 anti_theft_detection 技能：先用 /analogread 讀取光感測器，只有在夜間（亮度低於某值）才啟動視覺偵測，減少白天誤報。
8. **跨通道協作** 設計技能：當 MQTT 收到「有人回家」指令時，透過 Web 顯示歡迎訊息，並透過 Telegram 傳送「已開燈」確認。
9. **記憶管理技能** 新增一個 /autoCleanMemory 工具：當對話歷史超過 8000 字元時，自動總結最近對話並清除舊紀錄。
10. **語音控制完整鏈路** 測試並優化語音控制流程：用語音說「開燈」，驗證從語音轉文字 → 意圖理解 → 硬體執行的完整路徑，並記錄各步驟耗時。

B.3 專題級挑戰（適合期末專案或研究）

11. **教室環境監控系統** 整合 DHT11、光感測器與相機，每小時自動生成一份環境報告（溫度、濕度、光線狀態 + 照片），並透過 MQTT 與 Telegram 同時發送。
12. **多 Agent 協作** 使用兩台 fuClaw 裝置：裝置 A 偵測到人後，透過 /agentSendMessage 通知裝置 B 開燈並拍照。
13. **記憶體自動優化** 實作智慧記憶管理：在 /getMemory 顯示剩餘 heap 低於 30% 時，自動執行對話總結並清理歷史。
14. **安全監控儀表板** 擴展 Web 介面，新增即時顯示目前溫溼度、LED 狀態、最近 5 筆工具執行記錄的監控頁面。
15. **混合本地/雲端模式** 設計機制：當網路斷線時，切換到簡易本地規則模式（例如固定時間開燈）；恢復連線後自動同步狀態。

B.4 開放式研究挑戰

16. **本地模型整合** 探索如何在 Ameba Pro2 上整合小型本地模型作為 Gemini 的離線備援。
17. **視覺物件追蹤** 擴展 /vision 技能，讓 Agent 能辨識特定物件（例如「紅色物品」）並觸發對應動作。
18. **能源管理 Agent** 設計一個節能技能：根據光線與時間自動調整補光燈亮度，並記錄每日用電估計值。
19. **情感互動 Agent** 在 soul.md 中加入情緒辨識邏輯，讓 Agent 根據使用者訊息語氣調整回應風格。

20. **完整智慧家居系統** 整合 SG90（窗簾）、DHT11（環境）、LED（照明）、相機（監控），打造一個可透過單一指令控制整個房間的 Agent。

B.5 挑戰完成建議

- 每個挑戰都應包含：技能描述 → skill.md 內容 → 測試步驟 → 觀察結果 → 改進建議
- 鼓勵學生記錄開發過程中的錯誤與解決方法
- 優秀作品可上傳至 GitHub 作為作品集